

mySCDN - FlashCacheCDN

Iancu Stefan-Teodor

Faculty of Computer Science, Alexandru Ioan Cuza University of Iași, Romania

1 Introduction

1.1 General Vision

The proposal involves the creation of a CDN (Content Delivery Network) system aimed at delivering a service or transferring data as quickly as possible. Communication latency will be minimized, providing the user with a good application experience. Additionally, the goal of this project (acquiring hardware, deployment services, etc., enabling the entire system) is to generate minimal costs while delivering quality software services. The project solution proposes distributing multiple nodes in different geographical locations that will serve clients optimally.

1.2 Objectives

The project has the following objectives:

- Prioritizing the customer experience.
- Allocating the most convenient node for each client.
- Avoiding the risk of compromising the entire service and application by always having a backup option (if a node crashes, another node will provide the service with latency acceptable to the client).
- Managing concurrency as reliably as possible.
- Software optimization to minimize CPU usage.
- Gradual optimization of the caching system and resources generated by concurrency.
- Avoiding unforeseen cases and handling errors effectively.

2 Applied Technologies

- At the transport level, the project will use the TCP protocol throughout the system. Although it is more costly in terms of packet size and transmission time for a valid message exchange, it ensures that no packet will be lost or corrupted (internally). The project cannot afford, for example, to mistakenly allocate a node in America for a client in Madagascar due to the corruption of 3 bytes in a packet containing the IPv4 address of the node.

- To store information about the available cache on each node, the project will use the JSON format, allowing content to be deserialized into objects stored in memory. Server and client configuration files will be of type `.ini` because they are lightweight (as is the API) and easy to parse. Consequently, two open-source APIs for the C++ language were used, referenced in the bibliography notes.
- The architecture is designed for Unix-based systems. For network communication, standard C/C++ libraries for Unix sockets were utilized.
- Regarding concurrency, servers will implement multithreading, multiprocessing, and multiplexing depending on the case. For this, the standard libraries `<thread>` in C++ and `<unistd.h>` in C were used for management.
- The application logic was implemented using data structures provided by the standard C++ library and the OOP paradigm.
- CMake and bash scripts were used for easy builds and dependency installation.

3 Application Structure

The application architecture will be based on 4 entities: - Main server - Edge server - EDNS server - Client

A brief description of each:

- The main server will be responsible for storing all original resources, providing updates to edge servers, keeping track of cache and load on the edges, and transmitting it to the authoritative EDNS server.
- Edge servers will cache certain resources to serve clients while transmitting load information to the main server.
- The EDNS server will receive information about all edge servers and map them to make decisions based on cache, load, and geographic location. (it will also act as a load balancer)
- The client will make a request for the desired resource to the EDNS server, receive an IP address in response, and connect to the edge server with the specified IP address.

In general terms, the architecture of this type of application can be very well described through an image:



Fig. 1. CDN over the world

Now let's look at the next diagram (zoomed in):

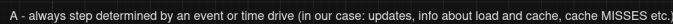


Fig. 2. CDN Architecture

4 Implementation Aspects

4.1 Application-Level Protocol and Innovative Code Sections

Client-EDNS Server Protocol:

- The request packet will consist of: packet length + client IP + resource + null byte
- The response will be 4 bytes representing the edge server's IP address

```
// Creating the packet
const size_t dns_packet_length = sizeof(size_t) + resource.length() + 1 + sizeof(in_addr_t);
// length header (8 bytes on x64 arch) + sender's public IPv4 (4 bytes) + len(request) + nullptr
char* dns_packet = new char[dns_packet_length];
const in_addr_t pub_ipv4n_addr = inet_addr(ep:public_ip.c_str());
memcpy(dns_packet, &dns_packet_length, sizeof(size_t));
memcpy(dest: dns_packet + sizeof(size_t), src: resource.c_str(), resource.length() + 1);
memcpy(dest: dns_packet + sizeof(size_t) + resource.length() + 1, &pub_ipv4n_addr, sizeof(in_addr_t));

// Sending the packet to the EDNS0 server
size_t bytes_sent = send(socket_fd, dns_packet, dns_packet_length, flags: 0);
```

Fig. 3. Example Client-EDNS server packet

Client-Edge Server Protocol:

- The request packet will consist of: packet length + resource + null byte
- The response will be packet length + response + null byte
- *Similar to Fig.3*

Edge Server-Main Server Protocol:

- The edge server will periodically transmit information about itself through a JSON file transfer or send a GET request to the server to request a resource in case of a cache MISS.
- The main server will send certain resources to the edge server along with a TTL (time to live) quantum.
- *Similar to Fig.3* but file transfer will be split into byte chunks.

Main Server-EDNS Server Protocol:

- The main server will transmit a JSON file with data about cache and load from edge servers.
- *Similar to Fig.3* but file transfer will be split into byte chunks.

```
{
  "10.126.86.215" : {
    "name": "edge-server-3",
    "load" : {
      "cpu_load": [
        0.22,
        0.05,
        0.02
      ],
      "memory_load": [
        376.5,
        900.0
      ],
      "disk_load": [
        1600.2,
        6384.6
      ]
    },
    "resources" : [". /test", ". /test_5_seconds"]
  }
}
```

Fig. 4. JSON data

```

class Cache
{
    string main_server_ip;
    int number_of_edge_servers;
    vector<Edge_server> edge_servers;
    void initialize(); // Initializing the cached data from the JSON config file
public:
    Cache(); // Will call initialize after allocating memory
    void update_mapping();

    // It is needed in order for the thread to access the actual object's instance function update_mapping()
    static void* update_mapping_entry_point(void* actual_object);

    // Decide which edge-server will pick based on the implemented algorithms
    in_addr_t decide(const char* resource, in_addr_t client_ip);
};

```

Fig. 5. Level 1 Deserialization

```

class Edge_server
{
    Load load;
    in_addr_t ip_address;
    string name;
    unordered_set<string> resources; // 0(1) lookups by using unordered_set
    // Compute the average load of the machine
    float get_average_load() const;

    // Estimating the geographical distance between the client and the edge-server
    float get_geo_distance_estimation_index(in_addr_t client_ip);
public:
    Edge_server(const Load& load, in_addr_t ip_address, const string& name, const unordered_set<string>& resources); // Constructor

    // Computing the final grade of the edge-server based on cached content, load and geolocation
    float compute_decide_grade(in_addr_t client_ip, const char* resource);
    string get_name() const;
    in_addr_t get_ip() const;
};

```

Fig. 6. Level 2 Deserialization

4.2 Concurrency

- **Main server** - At startup, it will perform **pre-forking** to handle the N edge servers, while the root process will communicate with the EDNS server. This approach was chosen because the project cannot afford for the Main server to fail if one process crashes. Even though the forking process is costly, it is done during initialization and does not affect concurrent communication.
- **Edge server** - It will contain two processes. One will communicate with the Main server, and the other will create a **thread pool** of size k (which will

be gradually optimized for accuracy). Each thread runs the same function to serve clients, invoking the `accept()` primitive inside a `mutex.lock()`. This approach is used because creating a thread on the spot for high traffic volumes can cost a few milliseconds and put a load on the CPU, even though threads are lightweight. This implementation will also depend on whether the deployment service supports **multithreading**.

- **EDNS server** - It will have two threads with simultaneous access to a **Cache** variable allocated on the heap. One thread will modify the variable when receiving updates from the Main server, while the other will handle clients in a **multiplexed** manner and access the variable to decide on the most suitable edge server. Of course, `mutex.lock()` will be used to protect against *undefined behavior* (e.g., one thread modifies while the other reads). This threading approach allows simultaneous access to **Cache**, and multiplexing is justified by reducing costs (CPU calls) and the lightweight nature of DNS server services.

- **Client** - Two processes. One will receive input and make requests to the EDNS server, then create a new process by calling `fork()` and redirect the request to the edge server, awaiting output. This approach allows a request not to be limited by the processing time of the previous (or other previous) requests.

In conclusion, the entire architecture of the developed CDN system can be encapsulated in the following detailed diagram (zoomed in):

- Adding a caching mechanism for lookups on the EDNS server.
- Developing an encryption tunnel for communication between servers, such as TLS.

6 References

- <https://edu.info.uaic.ro/computer-networks/>
- <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- <https://aws.amazon.com/what-is/cdn/>
- [https://en.wikipedia.org/wiki/Multithreading_\(computer_architecture\)](https://en.wikipedia.org/wiki/Multithreading_(computer_architecture))
- <https://www.youtube.com/watch?v=pdPSLm629yk&t=454s>
- https://en.wikipedia.org/wiki/Content_delivery_network
- <https://en.cppreference.com/w/cpp/thread/thread>
- <https://en.cppreference.com/w/cpp/thread/mutex>
- https://en.cppreference.com/w/cpp/container/unordered_set
- <https://github.com/nlohmann/json>
- <https://github.com/dzilles/configparser>